# Team#7 Final Report

## Development of AR Educational Chemistry Content


December 2025

# 1. Introduction

## Motivation

Chemistry is a subject that requires observing and understanding changes in matter; however, many chemical reactions involve invisible molecular processes or hazardous conditions that students cannot easily experience in typical classroom environments. Practical constraints such as limited laboratory space, expensive equipment, and strict safety regulations often restrict hands-on experimentation. As a result, chemistry education frequently becomes theory-centered and relies on memorization or passive video-based learning, which can reduce students' curiosity, engagement, and scientific inquiry skills.

To address these limitations, there is a need for a learning environment that allows students to safely and intuitively explore chemical phenomena while maintaining the procedural nature of real experiments. Advances in Augmented Reality (AR) technology provide an opportunity to create immersive educational experiences that bridge the gap between abstract chemical concepts and real-world laboratory practice.

## Proposal and Goal

This project proposes the development of an AR-based chemistry experiment application that integrates physical marker-based interaction with virtual experiment simulations. Students use handheld physical markers, recognized by smart devices, to manipulate virtual laboratory tools and trigger chemical reactions. This approach enables students to actively perform experiments in a safe and controlled virtual environment while experiencing a sense of realism and immersion.

**Development of Interactive Content:** We aim to implement three core experiment modules—Magnesium Combustion, Flame Test, and Iodine Clock Reaction—allowing students to safely visualize chemical reactions without the risk of explosions or burns.

**Enhanced Educational Features:** The application will include step-by-step text guides and post-experiment quizzes to support self-directed learning and assessment.

**Accessibility and Efficiency:** By utilizing Unity and AR Foundation, the project aims to ensure cross-platform compatibility (Android/iOS) and efficient development within limited resources. This ensures that the application can be widely deployed in schools or homes lacking expensive facilities, democratizing access to high-quality science education.

## Existing Approaches

Current chemistry education primarily relies on three distinct approaches, each presenting significant limitations:

**Textbook-based Learning:** Traditional theoretical learning focuses on memorizing experimental procedures and results through text and 2D images. While this method is cost-effective and time-efficient, it often results in rote memorization rather than deep conceptual understanding. Students remain passive recipients of information, lacking the opportunity to visualize abstract molecular interactions.

**Video-based Demonstrations:** To supplement textbooks, educators often utilize video materials. While these visuals provide a safer alternative to hazardous experiments and improve accessibility, they still offer a passive learning experience. Students merely observe the process without actively participating or making decisions, leading to limited engagement and retention.

**Physical Laboratory Experiments:** Conducting actual experiments is the most authentic method for kinesthetic learning. However, it faces substantial barriers, including safety hazards (e.g., chemical spills, fire), high operational costs, and time constraints in preparing and cleaning up materials. Furthermore, accurately tracking individual student performance and procedural adherence in a chaotic lab environment is a significant challenge for instructors.

In contrast, the proposed approach emphasizes physical interaction through AR markers that simulate real laboratory equipment operations. By promoting kinesthetic learning and supporting step-by-step experimental workflows, the system provides a more authentic and engaging learning experience. Furthermore, the integration of a web-based teacher dashboard allows for experiment data management and student performance tracking, extending the system beyond visualization into a comprehensive educational platform.

## Project main components

The project consists of four major components:

**(1) AR Client Application (Unity, C#)**

AR marker recognition

Experiment logic & visualization

UI/UX and interaction systems

**(2) Chemical Experiment Content**

Magnesium combustion experiment

Metal flame test experiment

Iodine clock reaction experiment

**(3) Backend Server (FastAPI)**

Student activity data management

Receiving and storing quiz/experiment results

API communication with the AR client and web frontend

**(4) Web Frontend (React)**

Teacher/admin dashboard

Displaying student experiment records

Basic UI components and authentication

# 2. Project Design and Implementation

- **User Feature**

<Teacher Features>

- Class Management System: Class code generation and management, student registration and class assignment
- Learning Monitoring Dashboard:
  - View all students' experiment performance records in the class
  - Check individual student scores and star acquisition status
  - View recent activity timeline
  - Statistics on average scores and total experiment count
- Web-based Management Interface: Real-time data visualization with frontend connected to FastAPI backend
- Class Code Sharing: Ability to display code via projector for sharing with students

<Student Features>

- AR Chemistry Experiments:
  - 3 types of experiment content (Magnesium Combustion, Metal Flame Test, Iodine Clock Reaction)
  - Interact with virtual experiment tools by manipulating physical marker cards
  - Character guide and step-by-step text instructions
- Self-directed Learning Support:
  - Post-experiment quiz system
  - Star and score acquisition system
  - Unlimited repetition of experiments
- Personal Learning Dashboard:
  - View total stars, total score, and completed experiment list
  - Check recent activity history
- Class Code-based Registration and Login

- **Main Feature**

**1. Physical Marker - 3D Model Mapping System (ARMarkerRegisterSystem)**

- Utilizes Unity AR Foundation's Image Tracking technology
- Built 1:1 mapping system between 14 markers and multiple 3D models
- Real-time virtual object synchronization based on physical marker movement/tilt
- String-based automatic mapping for improved development convenience
- Independent ARMarkerRegisterSystem for each experiment allows marker reusability

**2. Stage (Goal) Management System (UI Goal Manager)**

- Generic Abstract Class-based design enables creation of diverse experiment scenarios
- Step-by-step management:
    - Character dialogue system
    - Function execution at step start/end
    - Clear condition checking (isGoalCompleted delegate)
- Simple setup via drag-and-drop in Unity Inspector
- Efficient stage management using Dictionary

**3. Chemistry Experiment Physics Simulation**

- Liquid Interaction:
    - Utilizes LiquidVolumePro asset
    - Implements liquid pouring based on beaker tilt
    - Particle-based fluid movement simulation
- Chemical Reaction Visualization:
    - Utilizes Unity Particle System
    - Flame-metal contact recognition through Collision detection

    - Unique flame colors for each metal
    - Spectrum visualization through spectroscope UI
- Simplified Chemical Reaction Logic: Lightweight for mobile optimization

**4. Experiment Evaluation and Data Management System**

- JSON-based quiz question/answer management
- Automatic grading and result submission
- FastAPI + SQLite backend:
    - Store student experiment records (score, stars, timestamp)
    - RESTful API structure
    - Local server access through ngrok tunneling
- Real-time data visualization on web dashboard

**5. Marker Card Generation and Validation Tools**

- Custom Marker Generator:
    - 8x8 grid-based
    - Pseudo-random algorithm (random seed-based)
    - Automatic placement of 10-20 noise dots
- arcoreimg Recognition Rate Validation:
    - Generated 30 markers and evaluated on 100-point scale
    - Selected top 14 markers (all scored 100)
- Card Generator:
    - HTML/CSS/JavaScript-based
    - Automatic placement of marker name, image, and description
    - Generate 3-page sheet printable on A4 paper

- **Extension Feature(Features We Attempted but Couldn't Complete)**

1. **Advanced Physics Simulation**
    - Precise fluid dynamics-based simulation (currently using visual approximation)
    - Accurate reproduction of complex chemical reaction mechanisms (currently simplified logic)
2. **Markerless AR**
    - Hand tracking technology implementation
    - System capable of tracking objects without markers
    - Support for more dynamic manipulation activities
3. **Multi-platform Deployment**
    - iOS version build and deployment (currently Android only)
    - Web-based AR version (WebXR)
4. **Cloud Deployment**
    - Deployment on cloud services like AWS (currently using ngrok tunneling)
    - More stable server operation environment
5. **Advanced Learning Analytics Features**
    - AI-based learning pattern analysis
    - Personalized learning path recommendations
    - Recording and replay of experiment processes
6. **Additional Experiment Content**
    - More diverse chemistry experiment modules
    - Extension to other subjects (physics, biology, earth science)

## Technical Stacks

Development Hardware

- Test smartphones and tablets (iPhone, iPad, Galaxy, etc.)

Development Software

- Engine: Unity (AR Foundation package)
- 3D Modeling: Purchased from Unity Asset Store
- UI/UX Design: Figma
- Server & DB: ngrok (tunneling), FastAPI, SQLite
- Collaboration/Management: Git, GitHub, KakaoTalk, Discord

## Expected Challenges

<Technical Challenges>

1. Technical Limitations of Complex and Dynamic Manipulation
    - Constraints of image-based marker tracking: Camera must continuously illuminate markers
    - Recognition disconnection during rapid movements
    - Solution: Designed content focused on basic operations (movement, tilting)
    - Future improvement: Need to introduce markerless technology, hand tracking
2. Marker Occlusion by Hands
    - Recognition failure when hands cover patterns during flat marker manipulation
    - Solution: Created marker cards with handles to minimize interference
3. Trade-off Between Mobile Optimization and Simulation Precision
    - Limited mobile resources
    - Solution: Implementation centered on visual similarity instead of precise physics calculations/ Liquid movement approximation using shader graphs/ Particle-based simplified fluid movement/ Lightweight chemical reaction logic
    - Limitation: Unsuitable as a precision experimental tool for research purposes

4. Implementing Multi-marker-object Mapping System

- Unity AR Foundation's default functionality only supports single objects
- Solution: Developed custom ARMarkerRegisterSystem
- Managed 14 markers through string-based automatic mapping

5. Cross-platform Compatibility

- Differences between Android and iOS environments
- Solution: Support for both using Unity AR Foundation
- Actual deployment: Only Android completed due to time constraints

<Non-Technical Challenges>

1. Limited Time and Resources

- 4-month project period
- 4 team members (role division essential)
- Solution: Improved development efficiency by utilizing Unity game engine and existing assets

2. Difficulty in Experiment Content Planning

- Need for accurate understanding of chemical reactions
- Balance between educational value and fun
- Solution: Leveraged interdisciplinary team composition (Architecture + Software Engineering)

3. Lack of User Testing Environment

- Insufficient validation with actual students/teachers
- Solution: Internal team testing and feedback integration

4. Budget Constraints

- Total budget: 482,003 KRW
- Unity Asset Store asset purchase costs
- Solution: Utilized free assets + selective purchase of essential paid assets only

5. AR Marker Recognition Rate Optimization

- Need to ensure stability in various lighting environments
- Solution: Selected only 100-point markers using arcoreimg tool
- Generated 30 → Selected 14 for use

# 3. Design for the Proposed System

## Overall Architecture

- **Frontend Flowchart**

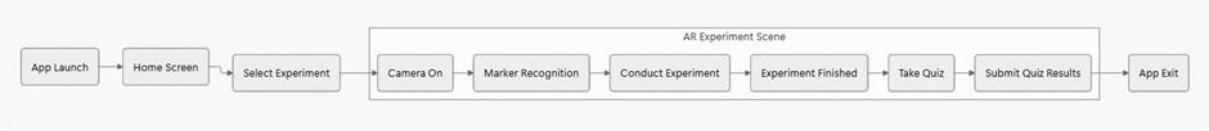



**Fig. 3-1 System Flow and Overall Architecture**

**Detailed Flow(Fig. 3-1) for Students:**

1. **App Launch**: Student launches the ChemAR application
2. **Home Screen**: Welcome screen with "START EXPERIMENT" button
3. **Select Experiment**: Choose from 3 experiments (Magnesium Combustion, Metal Flame Test, Iodine Clock Reaction)
4. **Camera On**: Grant camera permissions and activate AR camera
5. **Marker Recognition**: Scan physical marker cards with device camera
6. **Conduct Experiment**: Follow character guide and perform experiment steps
7. **Experiment Finished**: Complete all experiment stages
8. **Take Quiz**: Answer multiple-choice questions about the experiment
9. **Submit Quiz Results**: Enter class code and student information, submit to server
10. **View Results**: See quiz results and correct answers
11. **App Exit**: Return to home screen or close application

- **Backend API Specification**

**Fig. 3-2 API Summary Table**

| Domain | Resource | Method | Endpoint | Description |
|---|---|---|---|---|
| Admin | Teacher | POST | /admin/teacher | Create teacher account |
| Admin | Teacher | POST | /admin/teacher/link-class | Link teacher to class |
| Admin | Teacher | GET | /admin/teacher/classes/{user} | Get teacher's class list |
| Admin | Class | POST | /admin/class | Create class |
| Admin | Class | DELETE | /admin/class/{class_code} | Delete class |
| Admin | Experiment | POST | /admin/experiment | Add experiment |
| Admin | Experiment | DELETE | /admin/experiment/{...} | Delete experiment |
| Admin | Setup | GET | /admin/setup-data | Get all setup data |
| Admin | Setup | POST | /admin/reset-data | Reset data |
| Data | Submit | POST | /api/submit | Submit experiment results |
| Data | Query | GET | /api/dashboard/{code} | Get class dashboard |
| Data | Query | GET | /api/student/summary/{name} | Get student summary |

## Core API Details

**Fig. 3-3 Submit Experiment Results**

| Item | Detailed Content |
|---|---|
| Description | Stores experiment results performed by students. Creates new student if doesn't exist. |
| URL | POST /api/submit |
| Request | **Body (JSON)**<br>- class_code (string, required): Class code<br>- student_name (string, required): Student name<br>- experiment_title (string, required): Experiment title<br>- score (int, required): Obtained score<br>- stars (int, required): Obtained stars (0~3) |
| Response | - 201 Created: Success (saved)<br>- 404 Not Found: Experiment doesn't exist in the class |

**Fig. 3-4 Get Class Dashboard**

| Item | Detailed Content |
|---|---|
| Description | Retrieves all student list, average score, and individual student experiment performance for a specific class. |
| URL | GET /api/dashboard/{class_code} |
| Parameters | - class_code (string, required): Class code to query |
| Response | - 200 OK: Success (returns DashboardResponse) |

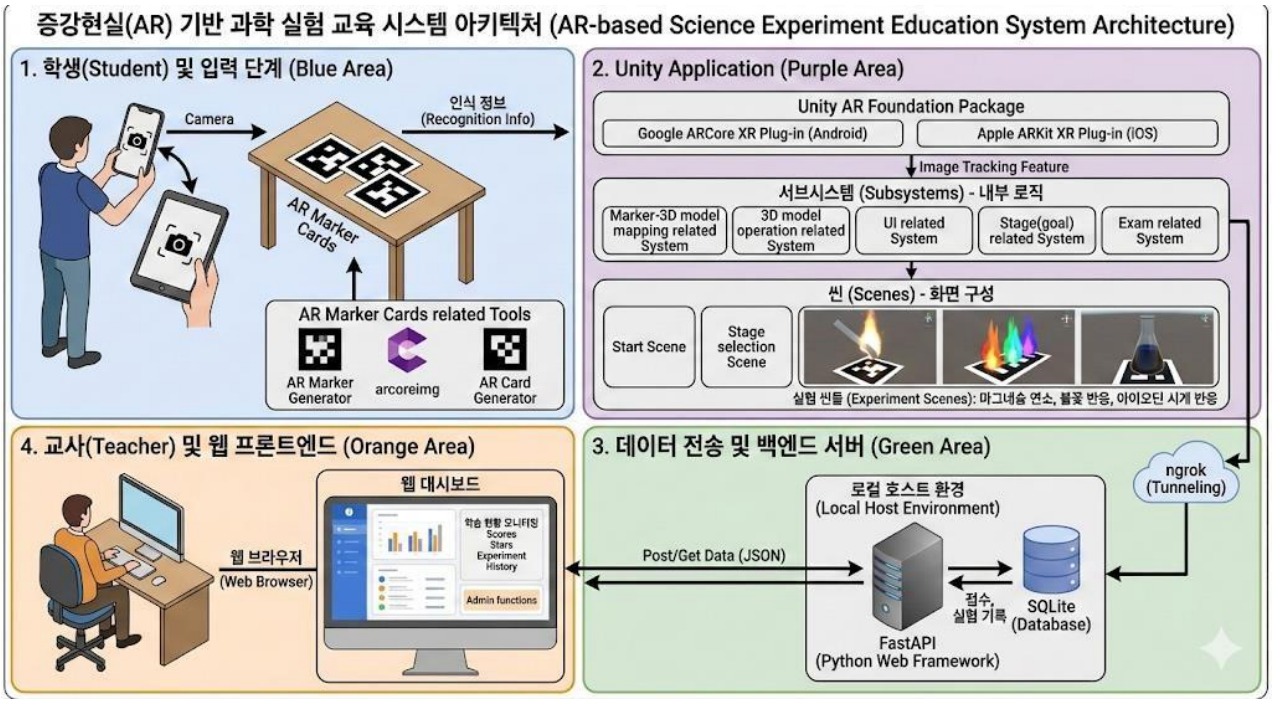
**Fig. 3-5 Get Student Detail Record**

| Item | Detailed Content |
|---|---|
| Description | Retrieves all experiment records for a specific student in detail. |
| URL | GET /api/student/summary/{student_name} |
| Parameters | - student_name (string, required): Student name |
| Response | - 200 OK: Success (returns StudentDetailResponse)<br>- 404 Not Found: Student not found |

**Fig. 3-6 Table 1 Get Teacher's Classes**

| Item | Detailed Content |
|---|---|
| Description | Retrieves list of all classes managed by a specific teacher. |
| URL | GET /admin/teacher/classes/{username} |
| Parameters | - username (string, required): Teacher username |
| Response | - 200 OK: Success (returns List[TeacherClassList])<br>- 404 Not Found: Teacher not found |

- **Architecture diagram**




- **Fig. 3-7 System Architecture**

## General Challenges

- **Frontend**

During the development of the frontend, specifically the AR application and web interface, the team faced and solved the following technical challenges:

- **Marker Recognition & Occlusion (User Interface & Interaction):**

◦ Challenge: Initial QR codes took too long to generate and had poor distance recognition. Additionally, standard paper markers were frequently occluded (blocked) by the user's hands during manipulation, causing tracking failure,.

◦ Solution: The team developed a custom marker generator based on an 8x8 grid and validated recognition rates using the arcoreimg tool,. Furthermore, they created card-type markers with handles to minimize hand interference with the marker pattern and improve usability,,.

- **Multi-Object Mapping System (Integration Logic):**

◦ Challenge: The default AR Foundation features focus on registering or managing a single virtual object, which limited the ability to map multiple physical markers to multiple 3D models in a 1:1 relationship,.

◦ Solution: They developed a custom system called ARMarkerRegisterSystem. This implements logic to automatically map marker names (strings) to 3D models using a dictionary, enabling multi-object interaction,,.

- **Mobile Optimization & Physics (Performance Optimization):**

◦ Challenge: Implementing precise simulations based on actual fluid dynamics within the limited resources of mobile devices could cause performance degradation.

◦ Solution: Visual effects like water wobbling based on tilt were implemented using Shader Graphs. The act of pouring liquid was handled using Particle Systems and simple data increment/decrement logic rather than complex physics calculations, ensuring visual similarity while reducing computational load,.

- **Limitations of Dynamic Manipulation (Responsiveness):**

  ◦ Challenge: Rapid shaking or moving of markers caused them to leave the camera angle or lose tracking connection.

  ◦ Solution: The content was designed around basic manipulation activities, such as moving markers closer or tilting them, to ensure the stability of recognition.

- **Backend**

Backend development focused on efficient data management and connectivity within a limited environment.

- **Server Accessibility & Deployment Environment (Managing Data Flow & Connectivity):**

  ◦ Challenge: Deploying via cloud services (like AWS) was cost-inefficient for the project's nature (intermittent use during class). However, a local server presented connection difficulties for student devices located on external networks.

  ◦ Solution: The team ran a FastAPI server on the teacher's local PC and utilized the ngrok tunneling service. This allowed secure access from external devices (student apps) to the local server without complex port forwarding configurations.

- **Database Design & Optimization (Optimizing Performance):**

  ◦ Challenge: Heavy database systems requiring separate server processes were cumbersome to install and operate.

  ◦ Solution: They adopted SQLite, a lightweight, file-based database that requires no separate server process. This was optimized to allow the teacher's PC to store and retrieve student data (scores, activity history) quickly and lightly.

# 4. Implementation

## Figma Design

The UI planner collaborated with the experiment process planner to design the interface required for the experiment. The process began with the UI planner creating sample images for the start and experiment screens using AI or design tools. Based on these samples, the UI developer built a prototype, which then underwent an iterative cycle of feedback and revision to finalize the design.

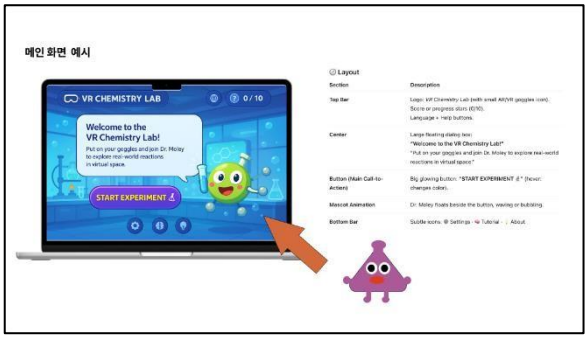



**Fig. 4-1. Initial UI Design Concept : Start Screen UI and Signature Character 1**

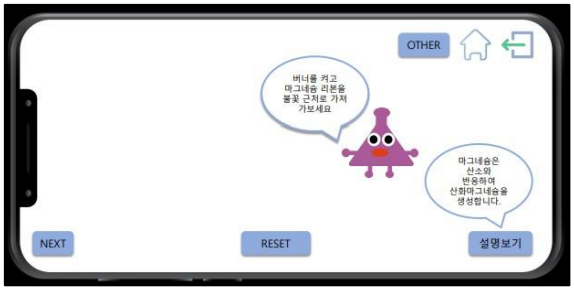



**Fig. 4-2. Initial UI Design Concept: UI Elements and Dialogue Planning for the Magnesium Combustion Experiment Screen**

The planner established the UI design concept for each screen (Fig. 4-1), which served as the blueprint for the developer to build the functional start screen (Fig. 4-4).

Additionally, the planning phase involved defining the necessary AR markers and the procedural flow for the experiments (Fig. 4-3). Based on this foundation, the specific experiment UI was designed, and the character dialogue was scripted (Fig. 4-2).

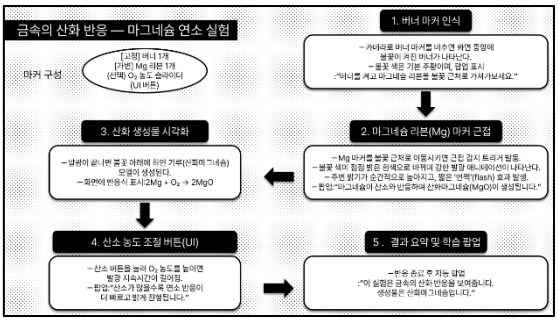



**Fig. 4-3. Planning for the Required AR Markers, 3D Models, and Step Flow Diagram**

Finally, the developer implemented an experiment screen prototype (Fig. 4-7) and shared it with the team to incorporate feedback through an iterative revision process.

## Main screens snapshots

- **Application UI**

① Start Screen and Experiment Selection Screen


**Fig. 4-4. Start Screen**


**Fig. 4-5. Experiment Selection Screen**

The Start Screen features introductory content about the application along with our project's mascot character. The screen background incorporates moving water droplets to capture students' attention and promote immersion. Pressing the 'Start Experiment' button on the screen transitions to the next view, the Experiment Selection Screen (Fig. 4-4).

On the Experiment Selection Screen, the UI at the top (emphasized by dots) enables users to quickly check the total number of experiments (stages) and the current stage sequence. Pressing the left and right arrow buttons displays the preceding and succeeding stages, respectively. Furthermore, the central UI prominently displays the experiment's theme for easy viewing, and the difficulty level is represented using three star images. As the AR experiment requires a QR image marker, necessary prerequisites (the QR marker) are explicitly listed in text for the user to identify them in advance. Pressing the 'To The Lab' button, located at the bottom center of the screen, transitions to the selected experiment (stage) view (Fig. 4-5).

② Magnesium Combustion Experiment

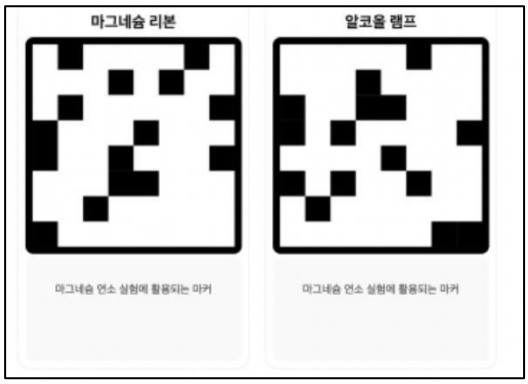



**Fig. 4-6. Markers Used in the Magnesium Combustion Experiment – Magnesium Ribbon Marker and Burner Marker**


**Fig. 4-7. Screen at the Start of the Experiment**

This experiment utilizes a Magnesium Ribbon AR marker and an Alcohol Lamp (Burner) AR marker (Fig. 4-6). When the experiment begins, the character in the bottom right assists by explaining the experiment, which the user can follow to proceed. The user can move the character freely by dragging, and can also fling the character in a specific direction using a simple swipe action. This feature was included to allow users to feel a sense of enjoyment simply by moving the character, encouraging better focus on the experiment. A Magnesium Ribbon 3D model appears on the Magnesium Ribbon AR marker, while a Burner 3D model appears on the Alcohol Lamp (Burner) AR marker (Fig. 4-7). The name of each 3D model appears above it in AR, and this name tag always rotates to face the user's camera (Fig. 4-7).


**Fig. 4-8. Scene of Magnesium Ribbon Combusting in the Flame**


**Fig. 4-9. Scene of Magnesium Ribbon Combusting and Turning into Powder**

When the magnesium ribbon is brought close to the flame, the part of the ribbon touched by the flame begins to turn white (Fig. 4-8, Fig. 4-9).


**Fig. 4-10. Situation where the Combustion Rate of the Magnesium Ribbon is Accelerated by**


**Fig. 4-11. Situation where the Magnesium Ribbon has Fully Combusted, Leaving Only White**

Increasing the oxygen concentration using the slider located on the left side accelerates the combustion rate of the magnesium ribbon, as shown in Fig. 4-10. Once the magnesium ribbon is fully consumed, only white powder remains (Fig. 4-11).

③ Metal Flame Reaction Experiment

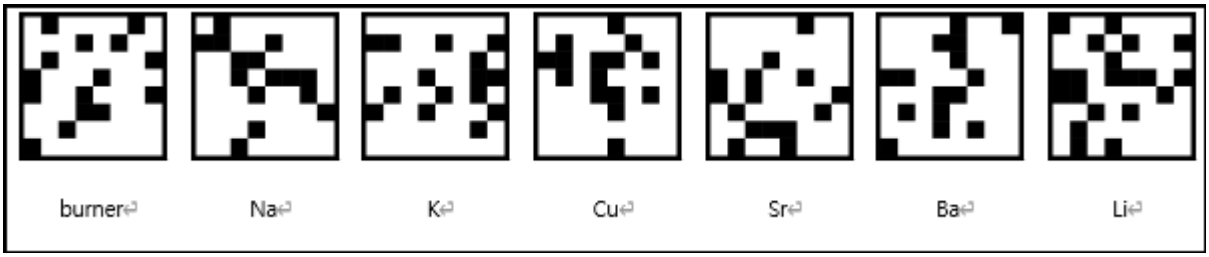



**Fig. 4-12. Markers Used in the Metal Flame Reaction Experiment – Burner Marker and 6 Types of Metal Element Markers**

This experiment utilizes an Alcohol Lamp (Burner) AR marker and six types of AR markers (Na, K, Cu, Sr, Ba, Li) (Fig. 4-12).


**Fig. 4-13. Screen at the Start of the Experiment**

A Burner 3D model appears on the Alcohol Lamp (Burner) AR marker, while cube-shaped metal models corresponding to their respective AR markers appear on the other metal AR markers (Fig. 4-13).


**Fig. 4-14. When Metal is Brought Close to the Flame – Cu Model Emitting Blue-Green Light**


**Fig. 4-15. When Metal is Brought Close to the Flame – K Model Emitting Purple Light**

When the metal 3D model is brought near the flame, a flame appears with the corresponding color of that metal's flame reaction (Fig. 4-14, Fig. 4-15).

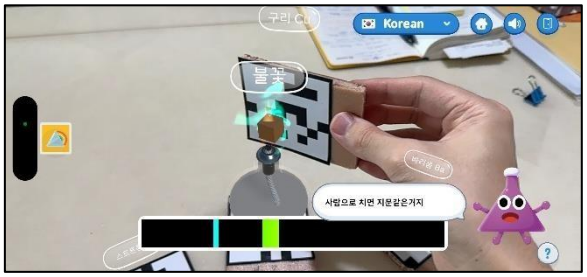



**Fig. 4-16. When Copper (Cu) is brought near the flame while the Spectrometer UI is enabled – The corresponding spectrum appears on the spectrometer**


**Fig. 4-17. When Potassium (K) is brought near the flame while the Spectrometer UI is enabled – The**

On the left side of the screen, there is a spectrometer toggle button that can be turned on or off. Pressing the spectrometer button to turn it on causes a UI to appear at the bottom of the screen to display the flame's spectrum. At this point, when the metal 3D model is brought close to the flame, a spectrum with the corresponding flame color for that metal's reaction appears in the spectrum display UI at the bottom (Fig. 4-16, Fig. 4-17).

④ Iodine Clock Reaction Experiment

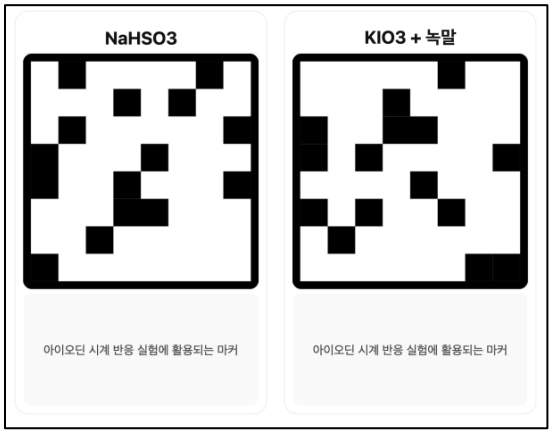



**Fig. 4-18. Markers Used in the Iodine Clock Reaction Experiment – Sodium bisulfite**


**Fig. 4-19. Screen at the Start of the Experiment**

This experiment utilizes a Sodium bisulfite (NaHSO3) AR marker and a Potassium Iodate (KIO3) + Starch AR marker (Fig. 4-18).

A 3D beaker model containing a blue solution appears on both the Sodium bisulfite (NaHSO3) AR marker and the Potassium Iodate (KIO3) + Starch AR marker (Fig. 4-19). Above the two 3D models, the name of the solution appears as an AR name tag.


**Fig. 4-20. Scene of Tilting One Beaker to Mix the Liquid with the Other Beaker's Liquid**


**Fig. 4-21. Scene of the Liquid Suddenly Changing to Purple after a Short Time**

When the 3D beaker model is tilted, the liquid inside the beaker flows out realistically. This allows the liquid to be mixed with the other beaker's liquid (Fig. 4-20). After the two liquids are mixed and time passes, the liquid suddenly changes color to purple (Fig. 4-21).

⑤ Basic Experiment UI


**Fig. 4-22. Double-Check Pop-up Window Displayed When the Exit Icon (Button) is Pressed ('Do you want to close the application?')**


**Fig. 4-23. CLEAR Pop-up Window Displayed Upon Completing the Experiment**

During the experiment, pop-up windows appear small in the center of the screen while the camera remains active (Fig. 4-22, Fig. 4-23). Pressing the Home or Exit button in the upper right corner of the experiment screen allows the user to navigate to the Start Screen or close the app. To prevent accidental exiting by the user, a double-check is performed via a pop-up window (Fig. 4-22). Upon completing the experiment, the CLEAR pop-up window appears. By clicking the desired button within this pop-up, the user can navigate to the Start Screen, proceed to solve test questions, or continue the experiment freely (Fig. 4-23).


If the user chooses to continue the experiment, a button labeled 'Solve Problems' appears at the top of the experiment screen, allowing them to go and answer test questions at any time (Fig. 4-24).

**Fig. 4-24. Screen with the 'Solve Problems' Button (Purple) at the Top**

⑥ Test UI

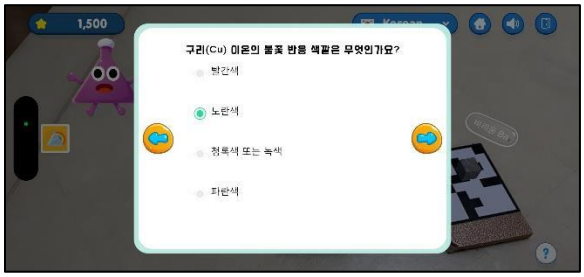



**Fig. 4-25. Pop-up Window for Solving Test Questions**


**Fig. 4-26. Screen of the Pop-up Window for Inputting Student Information**

When proceeding with the test, the questions are presented in a wide-screen multiple-choice format, where the user selects the correct answer out of four options (Fig. 4-25). Upon selecting an answer, an arrow button pointing to the right appears, which can be clicked to move to the next question. The user can click an arrow pointing to the left to return to the previous question. The user must solve all these multiple-choice questions. Once all questions are answered, a screen appears, as shown below, for inputting the user's information (Fig. 4-26).


**Fig. 4-27. Pop-up Window Displaying the Test Question Grading Results**

Once the user information is entered, the score is automatically calculated by comparing the user's selected answers with the actual answer key, and this score is transmitted to the server along with the user's information. After the transmission is complete, the user is intuitively shown whether their answers were right or wrong, and what the correct answers were if they made a mistake (Fig. 4-27).

- Web UI

① ChemAR Landing Page (Home)

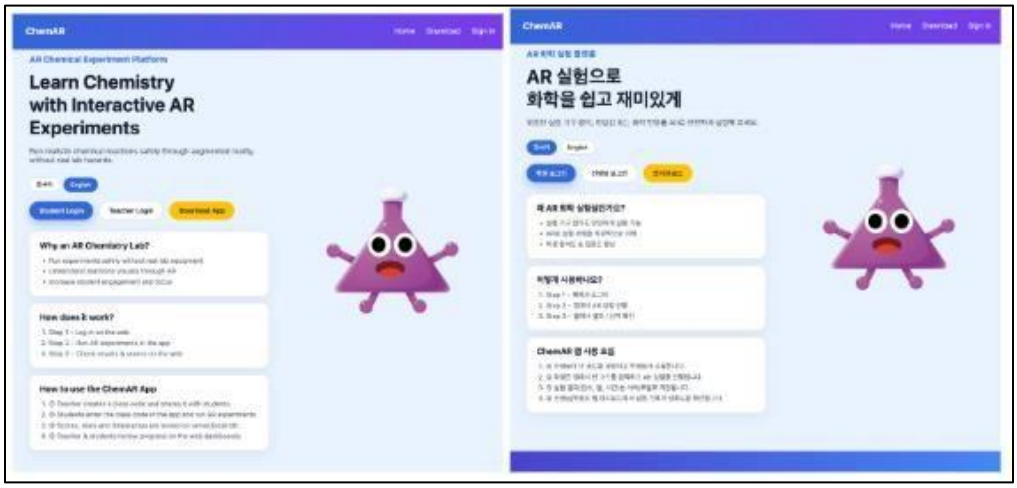



**Fig. 4-28. ChemAR Main Page (English and Korean versions)**

This page serves as the website's Home/Landing Page (Fig. 4-28). Both English and Korean versions are provided, and buttons for Student Login, Teacher Login, and App Download are displayed. The page also explains the necessity of the AR Chemistry Lab, the process and steps for using the application, and instructions on how to use the ChemAR application. The application's main character design is displayed on the right side of the page, with a subtle floating animation applied (moving slightly up and down) to enhance user interest and make the page more interactive.

② Login Page

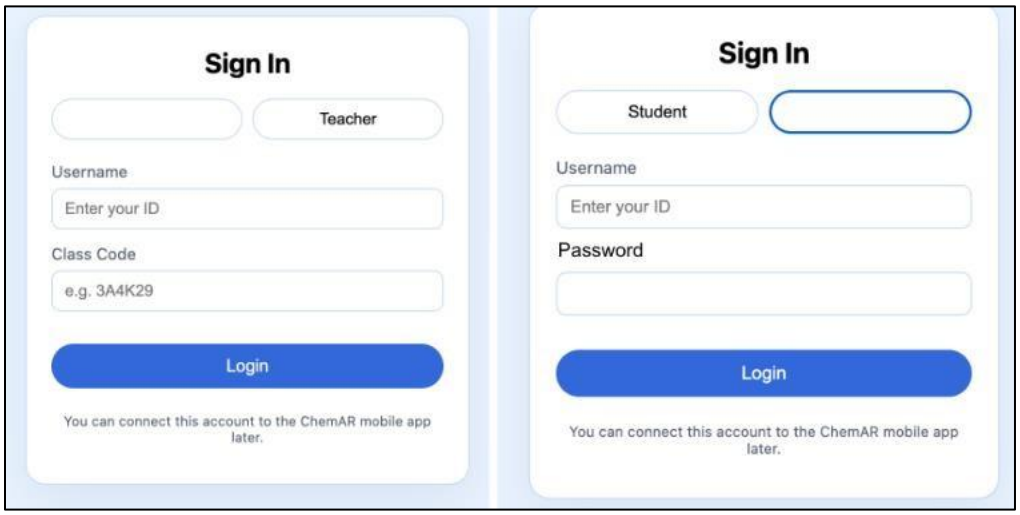



**Fig. 4-29 Login Page (Left – When Student is selected, Right – When Teacher is selected)**

When teachers log in, they must enter their name and password, while students log in by entering their username and class code (Fig. 4-29).

③ Student Dashboard

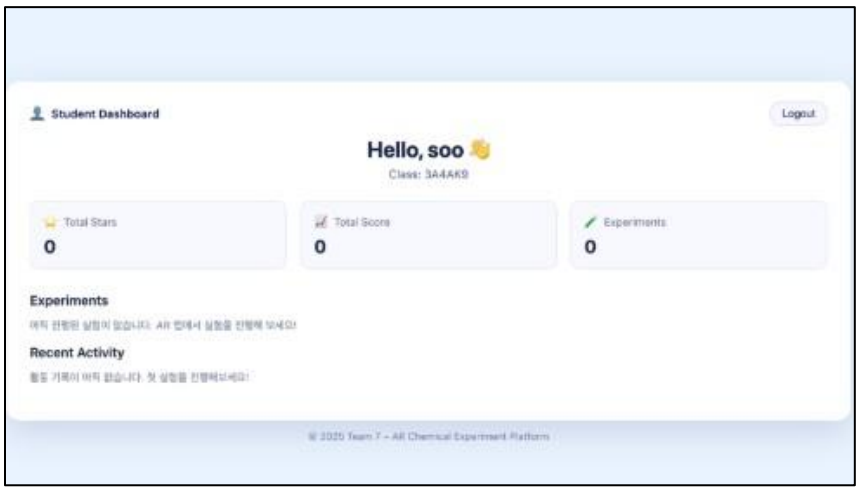



**Fig. 4-30. Dashboard Page Available for Students to View**

This page displays the student's total number of stars earned, total score, and a list of completed experiments. It also shows the student's recent activity history after using the ChemAR application (Fig. 4-30).

④ Teacher Dashboard

The class code created by the teacher, the number of students, the total number of experiments, the students' average score, and the total experiment count for that class are displayed. Below this, students' activity history is shown in a table format, displaying information such as student name, total score, number of attempts, and last activity time, based on a timeline (Fig. 4-31).

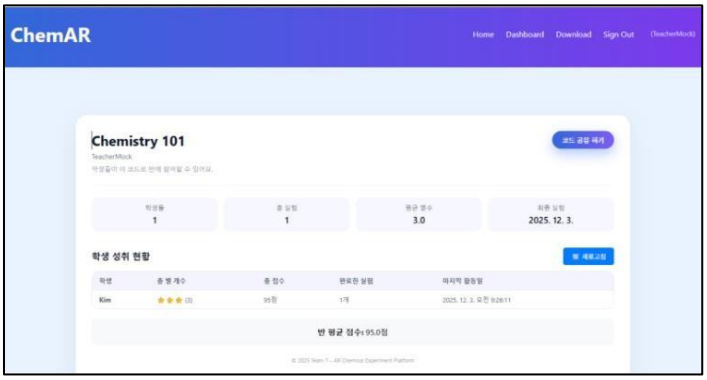



**Fig. 4-31. Dashboard Page Available for Teachers to View**

First, after the teacher logs in and shares the class code, the dashboard becomes available once students register by entering their name and the class code. When accessing the dashboard, a list of classes is displayed, and clicking on a specific class allows the teacher to view the student list and experiment results. When generating a class code, clicking the code initiates the server's generation of a random code, allowing students to use that code for registration.


The class code pop-up appears on the teacher dashboard when the teacher clicks the button on the right side while sharing the website with students. Students can use this class code to create an account and log in (Fig. 4-32).

**Fig. 4-32. Pop-up Window Displaying the Class Code Prominently**

⑤ ChemAR App Download Page

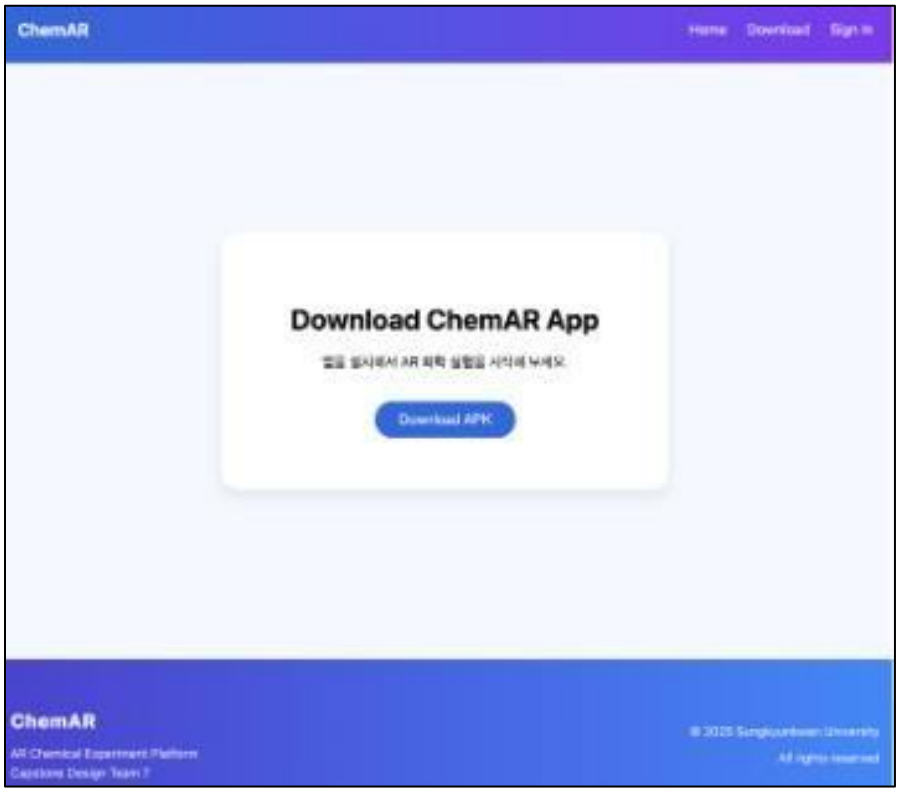



**Fig. 4-33. Page Where the ChemAR App Can Be Downloaded (Android only)**

This page appears when the &Download& text is clicked in the upper right corner of the website (if the app has not yet been downloaded) (Fig. 4-33). The ChemAR app can be downloaded from this page, and it is designed for use only on Android devices.

## Code snapshots

### Real-World Marker and 3D Model Mapping System

This system is responsible for mapping real-world markers to 3D models within a virtual environment. Since this feature is essential for all experiments, the goal was to develop it to be as intuitive and easy to use as possible.

The markers, which were created and verified using a custom-developed tool, were registered and stored in the Reference Image Library (RIL) (Fig. 4-34). This utilized the Image Tracking feature within Unity's AR Foundation package. This built-in feature provides the basic functionality (AR Tracked Image Manager) to recognize and track multiple images and place the registered 3D models at the location of the corresponding image. However, since the basic functionality only maps to a single 3D model, it was necessary to develop additional logic for mapping with multiple 3D models.

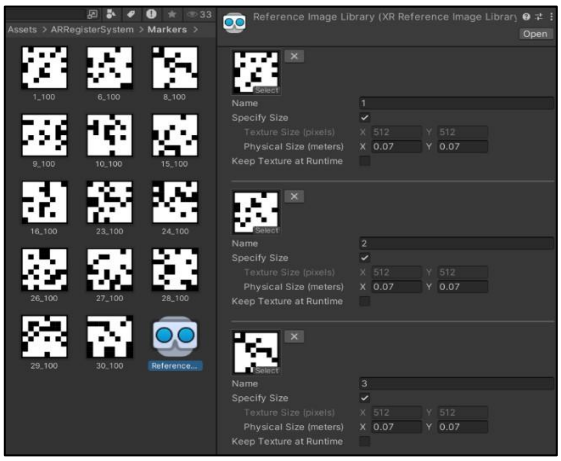



**Fig. 4-34. AR Foundation Package in Unity – Reference Image Library with Multiple AR**

Consequently, the Real-World Marker 3D Model Mapping System (ARMarkerRegisterSystem, Fig. 4-35) was developed. To enable easy utilization even without knowing the specific logic of the system, it was designed to automatically handle the mapping based on the name (string). A total of 14 markers were registered in the RIL, named with integers from 1 to 14. When a 3D model, named with the corresponding integer of the marker number (name) intended for use and mapping, is registered with the developed mapping system, the system automatically compares the strings and performs the mapping. Therefore, team members utilizing this system only need to develop the 3D model and its internal logic for the experiment, rename the model using the marker's number, and register it to establish the connection. Furthermore, once the mapping is complete through string comparison during the initialization process, the paired information is stored in a dictionary (Fig. 4-37) to allow for quick access during subsequent references (Fig. 4-36).

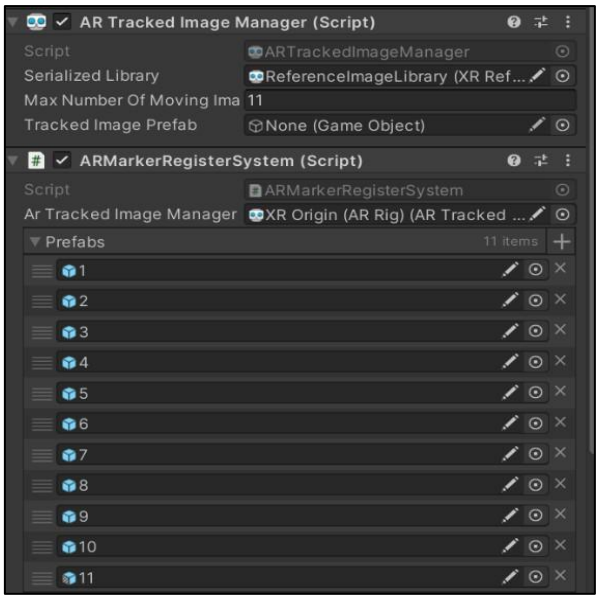



**Fig. 4-35. Real-World Marker 3D Model Mapping System (ARMarkerRegisterSystem) Various prefabs (3D models) are linked/connected to it**

```
public GameObject GetModel(string name)
{
    return _prefabDic[name];
}
```

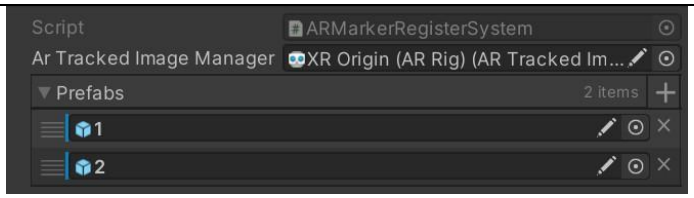
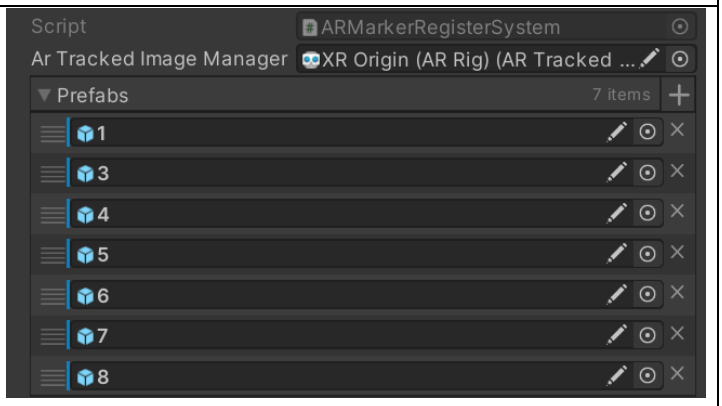
**Fig. 4-36. The GetModel function retrieving the AR model from the dictionary**

```
public class ARMarkerRegisterSystem : MonoBehaviour
{
    [SerializeField] ARTrackedImageManager arTrackedImageManager;
    [SerializeField] GameObject[] prefabs;
    private Dictionary<string, GameObject> _prefabDic = new Dictionary<string, GameObject>();

    private void Awake()
    {
        foreach (var prefab in prefabs)
        {
            var prefabName = prefab.name;
            var prefabObj = Instantiate(prefab);
            prefabObj.SetActive(false);
            _prefabDic.Add(prefabName, prefabObj);
        }
    }
```

**Fig. 4-37. The Awake function of ARMarkerRegisterSystem – Deactivating Prefabs and registering them in the dictionary**

**Fig. 4-38. AR Markers Registered in the ARMarkerRegisterSystem for Each Experiment**

| | |
|---|---|
| Magnesium Combustion Experiment |  |
| Metal Flame Test |  |
| Iodine Clock Reaction Experiment |  |

By choosing this approach, the same real-world markers can be reused by utilizing a separate ARMarkerRegisterSystem for each experiment. This means a total of 14 markers can be repeatedly utilized across different experiments. (Fig. 4-38)

```
private void OnChangeTrackingState(ARTrackedImagesChangedEventArgs eventArgs)
{
    foreach (ARTrackedImage trackedImage in eventArgs.updated)
    {
        if (trackedImage.trackingState == TrackingState.Tracking)
        {
            UpdateImage(trackedImage, IsActive: true);
        }
        else if (trackedImage.trackingState == TrackingState.Limited)
        {
            UpdateImage(trackedImage, IsActive: false);
        }
    }
}
```

**Fig. 4-39. Function distinguishing when an AR marker is recognized by the camera and when it is not**

```
private void UpdateImage(ARTrackedImage trackedImage, bool IsActive = true)
{
    string name = trackedImage.referenceImage.name;
    GameObject prefab = _prefabDic[name];
    prefab.transform.SetPositionAndRotation(trackedImage.transform.position, trackedImage.transform.rotation);
    prefab.SetActive(IsActive);
}
```

**Fig. 4-40. The Update Image function which activates/deactivates the AR model and synchronizes its position and rotation with the AR marker**

If the camera successfully recognizes the registered real-world marker, the corresponding 3D model is activated, and its position and rotation values are synchronized with the position and rotation values of the real-world marker within the camera view (Fig. 4-40). If the marker is not recognized by the camera, the corresponding 3D model is deactivated (Fig. 4-39, Fig. 4-40).

- **Stage (Goal) Related System: UI Goal Manager**

```
public abstract class UIGoalManager<T> : SerializedMonoBehaviour
    where T : Enum
```

**Fig. 4-41. UI Goal Manager Abstract Class**

```
public class Step
{
    /// <summary>
    /// The text to display on the button shown in the step instructions.
    /// </summary>
    [SerializeField]
    [TextArea(2, 10)]
    2 references
    public List<string> dialogText;

    /// <summary>
    /// Event that is invoked when this step starts.
    /// </summary>
    [SerializeField]
    1 reference
    public UnityEngine.Events.UnityEvent onStepStart = new UnityEngine.Events.UnityEvent();

    /// <summary>
    /// Event that is invoked when this step ends.
    /// </summary>
    [SerializeField]
    1 reference
    public UnityEngine.Events.UnityEvent onStepEnd = new UnityEngine.Events.UnityEvent();

    [SerializeField]
    1 reference
    public System.Func<bool> isGoalCompleted;
}
```

**Fig. 4-42. Step Class – Contains the character's dialog Text, and functions to run when each Step starts, ends, and a function to check the clear condition**

```
[Tooltip("Dictionary of Goals/Steps to complete as part of the user onboarding.")]
[SerializeField]
7 references
Dictionary<T, Step> m_StepDictionary = new();
```

**Fig. 4-43. Dictionary storing the Steps of a specific experiment**

This system manages the overall flow of an experiment. UI Goal Manager.cs (Fig. 4-41) is an Abstract Class using Generic (UIGoalManager<T>) that allows for the easy creation of diverse scenarios by inheriting from this class when making a new experiment scenario.UI Goal Manager.cs defines a Step, which represents a single stage within the experiment. It includes the character's dialog, a function to execute when the stage begins (onStepStart), a function to execute when the stage ends (onStepEnd), and a function to check the condition for when the stage should be cleared (isGoalCompleted) (Fig. 4-42). These elements can be simply connected and authored using drag-and-drop within the Unity Inspector (Fig. 4-52).

First, the detailed stages of each Step are stored in a Dictionary (Fig. 4-43). The Key for this dictionary is the stage name (defined using an enum structure, Fig. 4-49) which is defined in the experiment Class that inherits from UI Goal Manager.

```
protected virtual void PreprocessGoal()
{
    // 다음 step 시작 이벤트 호출
    if (m_StepDictionary.ContainsKey(m_CurrentGoal.CurrentGoal))
    {
        Debug.Log($"PreprocessGoal() - {m_CurrentGoal.CurrentGoal} step 시작 이벤트 호출");
        m_StepDictionary[m_CurrentGoal.CurrentGoal].onStepStart?.Invoke();
        // 다이얼로그 텍스트 설정 및 표시
        if (characterUI != null && stepList.ContainsKey(m_CurrentGoal.CurrentGoal))
        {
            // List<string>이면 새로운 ShowDialog 오버로드 사용
            if (stepList[m_CurrentGoal.CurrentGoal].dialogText != null)
            {
                characterUI.ShowDialog(stepList[m_CurrentGoal.CurrentGoal].dialogText);
            }
        }
```

**Fig. 4-44. The PreprocessGoal function executed when each Step starts – Internally Invokes the onStepStart function stored in the current Step**

```
protected virtual void ProcessGoal()
{
    // 현재 step 종료 이벤트 호출
    if (m_StepDictionary.ContainsKey(m_CurrentGoal.CurrentGoal))
    {
        Debug.Log($"ProcessGoal() - {m_CurrentGoal.CurrentGoal} step 종료 이벤트 호출");
        // 다이얼로그 숨김
        if (characterUI != null)
        {
            characterUI.HideDialog();
        }
        m_StepDictionary[m_CurrentGoal.CurrentGoal].onStepEnd?.Invoke();
```

**Fig. 4-45. The ProcessGoal function executed when each Step ends – Internally Invokes the onStepEnd function stored in the current Step**

```
void Update()
{
    // StartCoaching이 시작된 후에만 체크
    if (m_IsCoachingStarted && !m_AllGoalsFinished)
    {
        // IsGoal 같은 추상 메서드를 만들어 자식 클래스에서 비교 로직을 구현하게 함
        if (m_StepDictionary[m_CurrentGoal.CurrentGoal].isGoalCompleted?.Invoke() ?? false)
        {
            CompleteGoal();
        }
    }
}
```

**Fig. 4-46. The Update function executed every frame – Invokes the clear condition check function stored in the current Step every frame; if True, it executes the CompleteGoal function to move to the next Step**

The UI Goal Manager manages the execution of functions in the flow of each stage.

First, as shown below, the function that needs to be executed when the stage starts, which is defined in the Step, is executed by invoking the current stage's onStepStart within the PreprocessGoal function, which runs just before each stage begins (Fig. 4-44). Similarly, ProcessGoal executes the function that should run when the stage ends by invoking onStepEnd (Fig. 4-45). The Update function of the UI Goal Manager checks whether the current stage is completed or not by invoking the isGoalCompleted delegate (the current Step's completion condition) every frame (Fig. 4-46).

```
public class MetalFlameColorChangeManager : UIGoalManager<MetalFlameColorChangeManager.OnboardingGoals>
```

**Fig. 4-47. Metal Flame Color Change Manager Class inheriting UI Goal Manager (The Manager Class for the Metal Flame Test Experiment)**

Looking at the MetalFlameColorChangeManager class, which inherits UI Goal Manager (Fig. 4-47), we can see that it has an enum named OnboardingGoals defined within it (Fig. 4-49).

```
protected override void InitializeGoals()
{
    var readyGoal = new UIGoal<OnboardingGoals>(OnboardingGoals.Ready);
    m_OnboardingGoals.Enqueue(readyGoal);

    var findBurnerGoal = new UIGoal<OnboardingGoals>(OnboardingGoals.FindBurner);
    var moveMetalToFlameGoal = new UIGoal<OnboardingGoals>(OnboardingGoals.MoveMetalToFlame);
    var findSpectroscopeGoal = new UIGoal<OnboardingGoals>(OnboardingGoals.FindSpectroscope);
    var useSpectroscopeGoal = new UIGoal<OnboardingGoals>(OnboardingGoals.UseSpectroscope);
    var showResultGoal = new UIGoal<OnboardingGoals>(OnboardingGoals.ShowResult);
    var completeGoal = new UIGoal<OnboardingGoals>(OnboardingGoals.Complete);

    // 실험 순서대로 Goal 추가
    m_OnboardingGoals.Enqueue(findBurnerGoal);
    m_OnboardingGoals.Enqueue(moveMetalToFlameGoal);
    m_OnboardingGoals.Enqueue(findSpectroscopeGoal);
    m_OnboardingGoals.Enqueue(useSpectroscopeGoal);
    m_OnboardingGoals.Enqueue(showResultGoal);
    m_OnboardingGoals.Enqueue(completeGoal);
}
```

**Fig. 4-48. Initialization function that defines the Steps for this experiment and adds them to the Dictionary**

```
public bool IsFindBurnerGoal()
{
    var Burner = m_ARMarkerRegisterSystem.GetModel(((int)MarkerType.Burner).ToString());

    if (characterUI != null && characterUI.IsDoneAllDialog &&
    Burner != null && Burner.activeInHierarchy)
    {
        return true;
    }

    return false;
}
```

**Fig. 4-50. The clear condition check function for the FindBurner Step among the stages of the Metal Flame Test Experiment**

```
public void OnFindBurnerStepStart()
{
    if (characterUI != null)
    {
        var Burner = m_ARMarkerRegisterSystem.GetModel(((int)MarkerType.Burner).ToString());
        if (Burner != null)
        {
            characterUI.SetTarget(Burner.transform);
        }
    }
}
```

**Fig. 4-51. The function executed when the FindBurner Step among the stages of the Metal Flame Test Experiment starts**

```
public enum OnboardingGoals
    /// <summary>
    /// 0. 실험 준비 단계 (시작 전)
    /// </summary>
    1 reference
    Ready,

    /// <summary>
    /// 1. 버너 마커 인식
    /// </summary>
    1 reference
    FindBurner,

    /// <summary>
    /// 3. 금속을 불꽃에 이동하는 단계
    /// </summary>
    1 reference
    MoveMetalToFlame,

    /// <summary>
    /// 4. 스펙트로스코프 마커 인식
    /// </summary>
    1 reference
    FindSpectroscope,

    /// <summary>
    /// 5. 스펙트로스코프 사용
    /// </summary>
    1 reference
    UseSpectroscope,

    /// <summary>
    /// 6. 결과 및 요약 확인 단계
    /// </summary>
    1 reference
    ShowResult,

    /// <summary>
    /// 7. 모든 실험 완료 + 계속해서 즐기기
    /// </summary>
    1 reference
    Complete
}
```

**Fig. 4-49. Enum class defining the stages of the Metal Flame Test Experiment**

In the overridden InitializeGoals function, the stages of this experiment, defined as OnboardingGoals, are registered by being placed into a Queue in the actual experiment order (Fig. 4-48). Taking FindBurnerGoal, one of the Steps defined by OnboardingGoal, as an example, it is as shown in Fig. 4-49. The function to execute at the Start and End of the Step, and the function to check the Step's clear condition are defined as shown Fig,4-50 and Fig. 4-51.

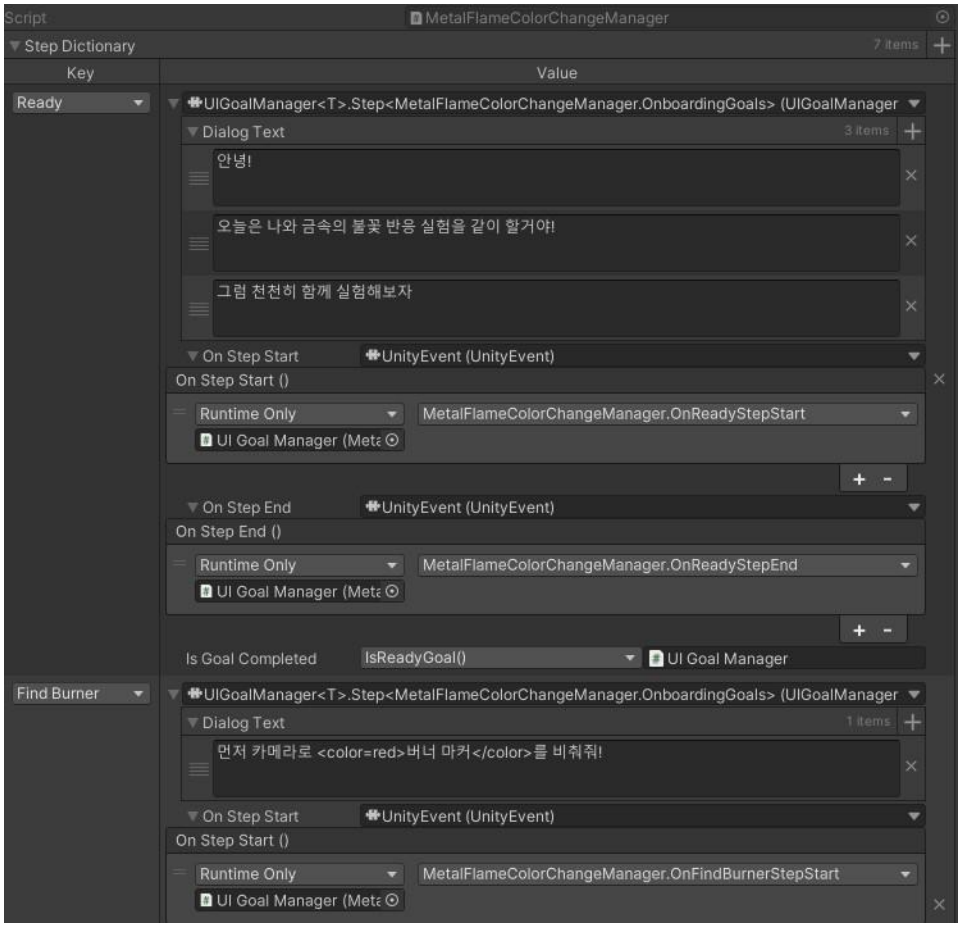



**Fig. 4-52. View of the MetalFlameColorChangeManager in the Unity Inspector**

If you check the component inheriting UI Goal Manager in the Unity Inspector, as shown in Fig. 4-52, you can see that for each Step, the stage name (Enum) is used as the Key to connect the character's dialog (Dialog Text), the On Step Start function, the On Step End function, and the Is Goal function to check the conditions for each stage.

- **Exam Related System**

When the AR experiment is finished, a pop-up window is presented to the user to solve exam questions. The exam questions and answers can be predefined in Json format within the Resources folder, as shown in Fig. 4-53. The relevant Json file is loaded using the Resources.Load function, as seen in Fig. 4-54, and the question and answer pairs are parsed and stored as shown in Fig. 4-55.

```
[
  {
    "question": "금속 이온의 불꽃 반응에서 나트륨(Na) 이온의 불꽃 색깔은 무엇인가요?",
    "options": [
      "빨간색",
      "노란색",
      "파란색",
      "초록색"
    ],
    "correctAnswer": 2
  },
  {
    "question": "구리(Cu) 이온의 불꽃 반응 색깔은 무엇인가요?",
    "options": [
      "빨간색",
      "노란색",
      "청록색 또는 녹색",
      "파란색"
    ],
    "correctAnswer": 3
  },
```

**Fig. 4-53. Json containing the question, options, and correct answer**

```
// 항상 Resources/ExamData 에서 로드 (모바일 빌드 포함)
TextAsset jsonFile = Resources.Load<TextAsset>($"ExamData/{jsonFileName}");
```

**Fig. 4-54. Function that loads a specific test paper json file from the ExamData folder**

```
// JSON 배열을 래퍼로 감싸서 파싱
string wrappedJson = "{\"questions\":" + jsonContent + "}";
ExamData examData = JsonUtility.FromJson<ExamData>(wrappedJson);

if (examData != null && examData.questions != null)
{
    questions = examData.questions;
}
```

**Fig. 4-55. Function that parses the Json into the QuestionData class**

```
public class QuestionData
{
    3 references
    public string question;
    6 references
    public List<string> options;
    2 references
    public int correctAnswer;
}

[System.Serializable]
2 references
public class ExamData
{
    2 references
    public List<QuestionData> questions;
}
```

**Fig. 4-56. QuestionData and ExamData – Can store the question text, a list of option texts, and the correct answer number**

Once all input is completed, the user's selected answers are compared with the actual answer key for automatic grading, and the resulting score is sent to the server along with the user's information. As shown in Fig. 4-57, a payload containing the student's input information and the exam score is prepared and transmitted to the web server using a Post format (Fig. 4-58).

```
var payload = new ExperimentWebClient.StudentExperimentResultPayload
{
    student_name = ResolveStudentName(studentInfo),
    class_code = studentInfo?.classCode,
    experiment_title = experimentWebClient.ResolveExperimentRoute(),
    stars = Mathf.Clamp(stars, 0, 3),
    score = normalizedScore
};
```

**Fig. 4-57. Payload containing the exam results and student information for the web server**

```
public async UniTask PostExperimentAsync(StudentExperimentResultPayload payload)
{
    if (payload == null) throw new ArgumentNullException(nameof(payload));

    var client = Fetcher;
    if (client == null) return;
    await client.Post<string>(experimentEndpoint, payload, DefaultHeaders);
}
```

**Fig. 4-58. The PostExperimentAsync function that asynchronously transmits the exam results and student information to the web server**

- **Web-Related System**

This system is the web-related system utilized by the educator. When developing the website, page-specific images, JavaScript, and CSS files were systematically organized (Fig. 4-59).

Each page has its own components and styles, while commonly used elements like the header and footer are structured as reusable JSX files. This structure was designed to make the website clean and scalable, and to ensure consistency in routing and UI/UX.

For the backend, Python and .txt files were used to store student and teacher records in a local environment, allowing the website to operate.

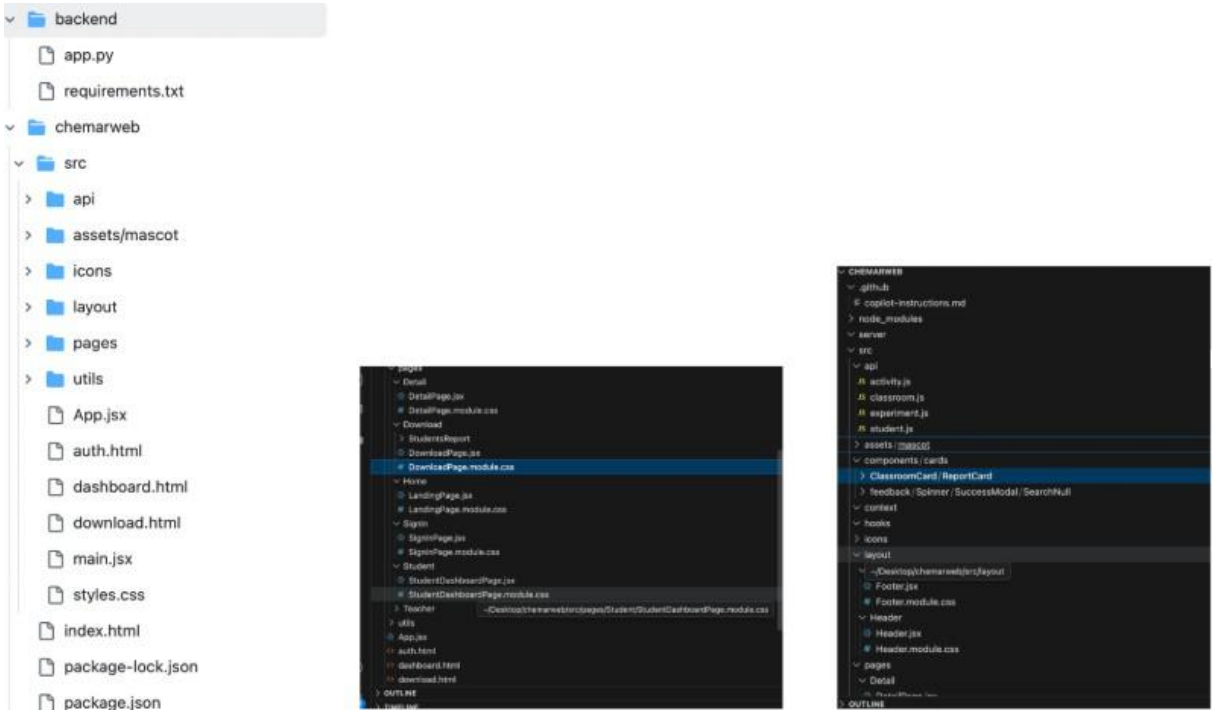



**Fig. 4-59. Web Development File Structure**

## Debugging and Testing

To ensure the Reliability and Stability of the system, systematic debugging and testing procedures were performed throughout the development process. In particular, considering the nature of the Augmented Reality (AR)-based system and the fact that users are from lower grades (or younger students), we focused on scenario-based functional testing and stress testing based on unexpected situations. Also, we tested various times for the beaker liquid chemicals, and fire motions to move correctly.

### (1) Testing Methodology and Process

#### (a) Module/Unit Testing and Integration Testing

- **Module Testing:** Each time an individual module (Scene, Component) constituting a feature was completed, a unit test was performed to confirm that the function of that module was operating correctly according to the design. For example, core functions such as marker recognition, 3D model loading, and animation playback within a specific scene were verified to run independently without errors.
- **Integration Testing:** After completing testing on individual modules, they were combined and integrated into the entire system. We then checked for issues in the interaction and Data Flow between the modules. This process helped to identify and correct potential bugs early that arose when features were connected.

#### (b) Scenario-Based Functional Testing

- **Developer Demonstration:** Following the integration testing phase, the development team directly tested the system by demonstrating it according to the final user's (End-User's) expected scenarios.
    - **Normal Flow Simulation:** We followed the typical usage flow according to user guidelines (App launch → Marker recognition → Content interaction → Exit) to verify that all features functioned as expected.
    - **Usability and Flow Review:** Beyond just functional errors, we subjectively evaluated aspects of User Experience (UX), such as the smoothness of screen transitions and the intuitiveness of the interface, to identify areas for improvement.

(2) **Abnormal Operation and Stress Testing**

To ensure that the system operates robustly even under abnormal input or environmental conditions, we conducted advanced testing that simulated exception scenarios.

**(a) User Abnormal Behavior Simulation**

- **Random Input and Exploration (Ad-Hoc Testing):** We tested unexpected user behavior by randomly and quickly tapping buttons or using features in an unusual order to see how the system responds.
- **AR Environment Interference-Camera Shaking:** With the marker recognized, we quickly shook or rapidly moved the device to check how stably the AR content remains fixed (Tracking) despite environmental changes.

**(b) Compound Environment and Overload Testing**

- Multiple Marker Recognition: Assuming a situation where the system must handle several AR markers simultaneously, we checked system resource consumption and stability by exposing multiple markers on a single screen concurrently or by rapidly and sequentially repeating marker recognition and release.
- Build Environment Testing: Since both the Android and iOS environments are commonly used in Korea, we built and verified the operation of the application in both environments and conducted subsequent testing.

# 5. Expected Result

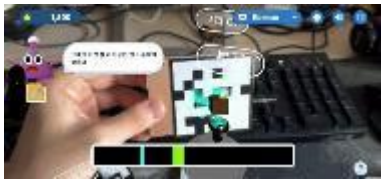
**(1) Cross-Platform Application(Fig. 5-1)** : Completion of a stable AR application compatible with both Android and iOS using Unity and AR Foundation.

▪ **Fig. 5-1. AR Chemistry Experiment Application running on iOS and Android environments**

| AR Chemistry Experiment Application Startup Screen | iOS Environment (iPhone 14 Pro) Compatibility Check | Android Environment (Galaxy S25+) Compatibility Check |
|---|---|---|
|  |  |  |

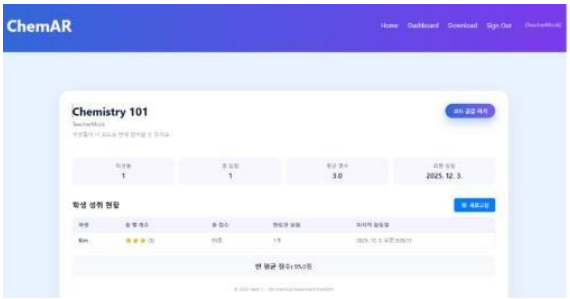
**(2) Three Core Experiment Modules(Fig. 5-2):** Full implementation of Magnesium Combustion, Flame Test, and Iodine Clock Reaction with tangible marker interactions.

▪ **Fig. 5-2. 3 chemical experimental content modules**

| magnesium combustion experiment | metal flame reaction experiment | iodine clock reaction experiment |
|---|---|---|
|  |  |  |

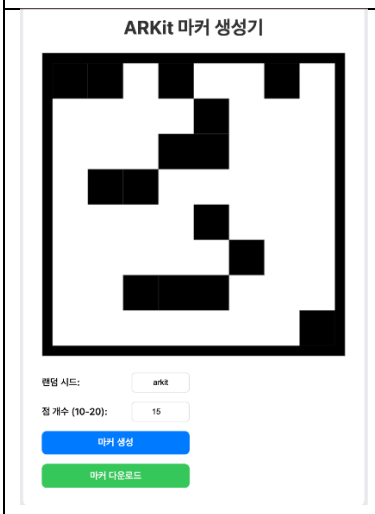
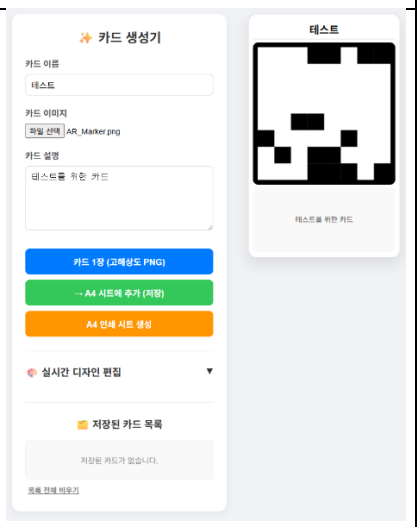
**(3) Integrated Education System(Fig. 5-3):** Establishment of a comprehensive system featuring character guides, quiz modules, and a database for tracking and analyzing student learning logs.

**Fig. 5-3. Quiz module & Dashboard for Analyzing**

| Quiz ModuLes (Send Data to server) | Web Dashboard for analyzing student's data |
|---|---|
|  |  |

**(3) Establishment of AR Marker Production Process(Fig. 5-4):** Establishment of a standardized AR Marker Production and Verification Process utilizing open-source software and custom tools.

**Fig. 5-4. AR Marker Production related tools**

| AR Marker Generator | AR Marker Verification (arcoreimg : opensource) | AR Marker Card Generator |
|---|---|---|
|  |  |  |

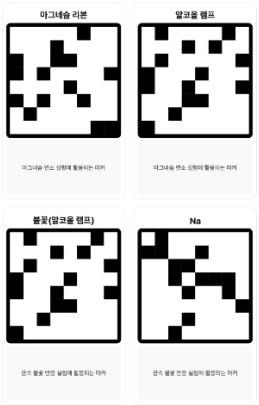
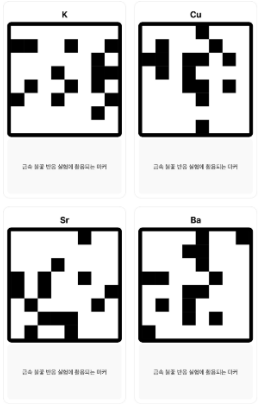






**(4) Outputable AR Marker Card Set(Fig. 5-5):** Provision of a ready-to-print PDF file containing the specific marker designs required for experimental interworking, ensuring immediate usability for end-users.

**Fig. 5-5. AR Marker Card Set**

| Page 1 | Page 2 | Page 3 |
|---|---|---|
|  |  |  |

# 6. Limitations and Discussions

**(1) Technical limitations of marker manipulation**

Current image-based marker tracking technology has a limitation that the camera must continuously illuminate the marker. As a result, when implementing a dynamic experiment in which the experimental tool must be rapidly shaken or manipulated in a complicated manner, there is an inconvenience that the user has to continuously adjust the camera position due to the phenomenon of leaving the camera angle or disconnecting the recognition connection. Therefore, in this task, for the stability of recognition, the content was composed mainly of basic manipulation activities such as simply moving the markers close to each other or tilting them at a certain angle. In the future, it is necessary to increase the degree of freedom of operation by introducing markerless technology or hand tracking technology that can track objects without markers.

**(2) Hand masking and marker card simplicity**

In the case of using an existing flat paper marker, there was a frequent problem in that recognition failed by covering the image pattern of the marker with the hand while the user held or manipulated the marker. To improve this, in this task, a marker card that allows the user to hold and manipulate the handle part, not just a printout, was separately produced. Through this, a more stable experimental environment was established by minimizing the interference phenomenon in which the hand obscures the core pattern of the marker and increasing the convenience of operation. However, in the case of a simple flat card, activities such as pouring liquid led to unintuitive manipulation. Therefore, to improve this, it is necessary to consider methods such as attaching markers to real models such as beakers rather than simple cards.

**(3) Simplification of Physical Operations and Limitations of Simulation Precision for Mobile Optimization**

In order to secure stable performance within the limited hardware resources of mobile devices, an optimization technique emphasizing visual similarity was applied instead of precise simulations based on actual fluid dynamics. For example, the liquid movement inside the beaker did not calculate the interaction of a large number of particles, but used a shader graph to implement a visual effect as if the water surface was shaken according to the device's slope value. In addition, the act of following a solution was also simplified by creating particles with liquid attribute data at the snout position and reducing the liquid data value inside the beaker when tilted above a critical angle, rather than the movement of physically connected fluids. Chemical reactions also have simplified logic for performance and stable implementation. Therefore, this content is suitable for educational purposes to help understand the concept, but there is a limit to using it as a precision experiment tool for research purposes that should reflect even fine variables of actual natural phenomena. To improve this, optimization and coordination by experts in chemistry subjects are needed.

# 7. Related Works

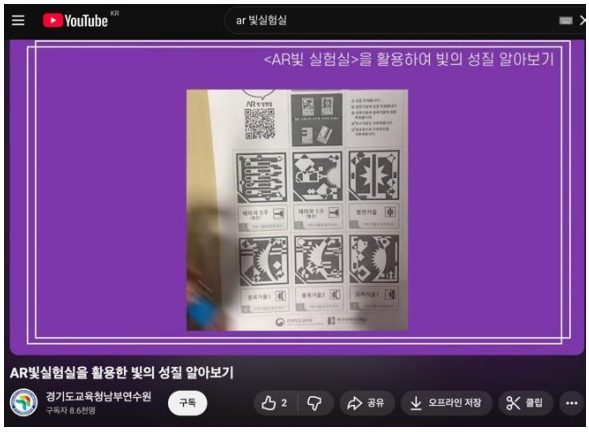



**Fig. 7-1. AR Light Lab: AR Markers**


**Fig. 7-2. AR Light Laboratory : AR Light Reflection Experiment**

This project was conceived with inspiration from the "AR Light Laboratory"(Fig. 7-1, 7-2), an application developed in 2007 with the support of the Korea Foundation for the Advancement of Science and Creativity and the Ministry of Science, ICT and Future Planning. The "AR Light Laboratory" offers the distinct advantage of allowing anyone to easily conduct light-related experiments without the need for expensive equipment such as lasers or lenses. However, the application is limited to observing results by placing a marker on a surface and making minute adjustments. This passive and restricted method of interaction limits the learner's ability to experience the sense of realism associated with directly manipulating experimental tools.

In contrast, the "AR Chemistry Experiment" proposed by our team provides a self-directed learning environment in which a virtual character acts as a tutor, guiding the learner through the experiment step-by-step. The most significant differentiator is the sophistication of the interaction method. Learners go beyond simply moving a marker on the floor; they physically hold and manipulate the marker as if they were handling actual experimental tools (e.g., beakers). This direct manipulation allows learners to experience a much higher level of immersion and educational effectiveness.

To support systematic educational activities that transcend mere experience, we have integrated an evaluation and management system. After completing an experiment, students can assess their comprehension by solving pre-registered quizzes. Furthermore, educators can access student quiz data and analyze academic achievement directly through a web browser without the need to install separate software. This significantly enhances the convenience and usability of the system in actual educational settings. Also, by refining and disseminating the implementation methodology for AR educational content through this project, we have established a technical foundation that allows anyone to easily create and add new AR content in the future. This is significant as it presents the potential for development into a sustainable educational platform.

# 8. Conclusion

This project was initiated to address the intrinsic limitations of traditional chemistry education specifically, the safety hazards of hazardous materials, the high cost of equipment, and spatial constraints that often reduce experimental learning to passive observation. The primary objective was to develop an "Interactive AR Chemistry Experiment Application" that bridges the gap between abstract theoretical knowledge and physical practice by combining Augmented Reality (AR) technology with tangible marker-based interactions. The goal of the project is to bridge the gap between abstract chemical theory and practical experimentation by providing safe, visible, and engaging learning experiences.

We utilized Unity and AR Foundation to build a cross-platform solution compatible with both Android and iOS, ensuring high accessibility. The project successfully implemented three core experimental modules: Magnesium Combustion, Flame Test, and Iodine Clock Reaction. A key technical differentiator is the adoption of a Tangible Interface, where printed marker cards serve as physical controllers, allowing users to perform multi-step procedures directly. Unlike existing commercial apps that rely on simple visual observation, our system provides comprehensive support for the entire educational cycle. This includes friendly character-led explanations, step-by-step procedural guides, post-experiment performance assessments, and features to view and analyze accumulated exam data. This holistic approach—from preparation to execution and analysis—sets this application apart as a unique educational platform.

The most significant educational contribution of this project is the realization of Kinesthetic Learning within a digital environment. The process of physically moving the body and manipulating markers to trigger virtual reactions effectively simulates the "hands-on" experience of a real laboratory. This physical interaction is superior to mere viewing in reinforcing procedural memory and aiding conceptual understanding. Furthermore, by providing a "safe failure" environment where students can undergo trial and error without the fear of chemical spills or explosions, the system establishes a foundation for students to immerse themselves in scientific inquiry without hesitation.

This project is expected to contribute to the Democratizing of Science Education. By replacing expensive laboratory infrastructure with scalable software and simple printed markers, it allows schools and households with limited budgets or geographic constraints to access high-quality experimental education, thereby helping to bridge the educational gap.

In conclusion, this AR application stands as an empirical example of how immersive technology can revolutionize STEM education. By resolving the dilemma between "observation" and "participation," and offering a system that is safe, cost-effective, yet highly interactive, it is poised to become a standard educational tool for cultivating future scientific talent, especially when integrated with data-driven assessment models in the future.

# 9. References


순순스튜디오. (2023a, 07. 04.). *Unity AR Foundation* 강좌 - 다중 이미지 트랙킹 [Video]. YouTube. https://www.youtube.com/watch?v=JaxT9pDxfBs

순순스튜디오. (2023b, 07. 04.). *Unity AR Foundation* 강좌 - 싱글 이미지 트랙킹 [Video]. YouTube. https://www.youtube.com/watch?v=KHhxgGTDHms

Google. (n.d.). *The arcoreimg tool*. Google Developers. https://developers.google.com/ar/develop/augmented-images/arcoreimg

Unity Technologies. (n.d.). *AR Foundation Manual*. Unity Documentation. https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@4.2/manual/index.html

경기도교육청남부연수원. (2020. 11. 30.). *AR*빌실험실을 활용한 빛의 성질 알아보기 [Video]. Youtube. https://www.youtube.com/watch?v=qjeTiulv1Pg

EBSi. (2022. 8. 18). 농도 및 온도와 반응 속도 | *Real* 과학 실험 [Video]. Youtube. https://www.youtube.com/watch?v=lArZvzjps28